

Conceptos y Paradigmas de Lenguajes de Programación - 2025 - Primer Parcial 1ra Fecha. T1
25/04/2025

HOJA 1/3

Realice el parcial con lapicera de otra forma se desaprobará el/los ejercicio/s.
Se considera presentismo cuando se realiza completamente un ejercicio.

Legajo: 2306110					Corrigió:
Apellido y Nombres: Guaymas Matías Julián					Linone
1	a	B-	b	B.	
2	a	B	b	B	c B d B.
3	a	B.	b	B	
4	a	B	b	B	c B-
Resultado Final					Aprobado

Ejercicio 1

a) (Pts.25) Defina, utilizando EBNF, la gramática para una función de C. Puede considerar (además de otros) un ejemplo como (no implemente solo el ejemplo):

```

tipo nombre_funcion (lista parámetros)
{
    instrucciones
    return xxxx (valor/variable del tipo de la función)
}
    
```

b) (Pts. 5) Realice el diagrama sintáctico de la gramática correspondiente.

Ejercicio 2

Sea el siguiente código en Ada, indique para todos los identificadores

- a) (Pts. 5) Su tipo de ligadura con l-valor. b) (Pts. 5) Su r-valor al momento de declaración (inicialización).
c) (Pts. 5) Tiempo de vida . d) (Pts. 5) Alcance

1. with text_io; use text_io;
2. Procedure Main is;
3. type arreglo_enteros is array(integer range <>);
4. b, m, t:integer;
5. vec:arreglo(1..100);
6. x: constant integer:=5;
7. Procedure Proc is;
 - 7.1. type puntero is access integer;
 - 7.2. vec2:arreglo(0..x);
 - 7.3. t: puntero;
 - 7.4. begin
 - 7.4.1. m:=4;
 - 7.4.2. vec2(m):= vec2(1) + vec(4);
 - 7.4.3. t:= new puntero;
 - 7.4.4. free t;
 - 7.5. end;
8. begin
9. m:=5;
10. Proc;
11. b:= m + 2;
12. end.

Realice este ejercicio sobre esta misma hoja.

Identif	L-value	R-Value	Alcance	T.V.
Main (2)	—	—	3-12	2-12
b(4)	autónoma	basura	5-12	2-12
m(4)	autónoma	basura	5-12	2-12
t(4)	autónoma	basura	5-7.3 8-12	2-12
vec(5)	autónoma	basura	6-12	2-12
x(6)	autónoma	basura	7-12	2-12
Proc(7)	—	—	7.1-12	7-7.5
vec2(7.2)	semiautónoma	basura	7.3-7.5	7-7.5
t(7.3)	autónoma	nil	7.4-7.5	7-7.5
t ¹	dinámica	basura	7.4-7.5	7.4.3-7.4.4

Realice el parcial con lapicera de otra forma se desaprobará el/los ejercicios.
Se considera presentismo cuando se realiza completamente un ejercicio.

Ejercicio 3

Sea el siguiente programa escrito en Pascal like, realice la ejecución del programa presentado siguiendo:

a) (Pts. 20) Cadena estática. b) (Pts. 5) Especifique cuáles son los campos que componen un registro de activación y para qué sirve cada uno de ellos.

El lenguaje evalúa expresiones de izq. a der.

```
Program MiPrograma
var i: integer;
var x: integer;
var a: array[1..6] of integer;
```

Procedure Tres

```
begin
  i := i + 2;
  x := x - 1;
end
```

Procedure Dos

```
var x: integer;
Function Fun: integer;
begin
  i := i - 1;
  return x + 1;
end
begin
  x := 7;
  a[i] := Fun();
end
```

Procedure Uno

```
var i: integer;
begin
  for i:=1 to 6 begin
    a[i] := i * 2;
  end
  i := 6
  Dos();
  Tres();
  write(i);
end
```

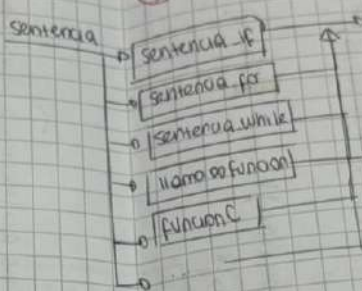
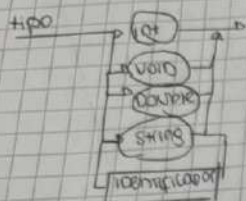
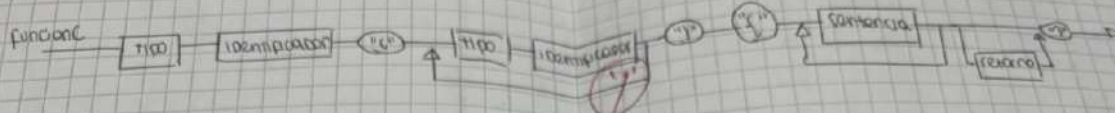
begin

```
i := 3;
x := 5;
Uno();
write(i);
write(x);
for i := 1 to 6 begin
  write(a[i]);
end
end
```

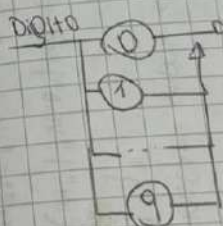
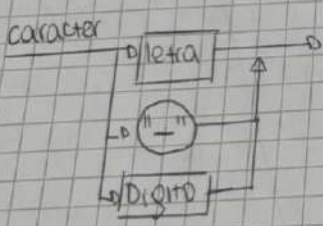
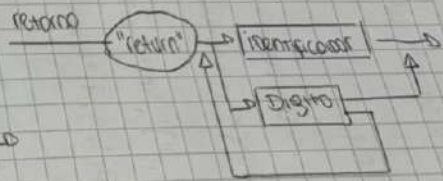
Ejercicio 4

- a) (Pts. 10) Indique dónde se almacenan las variables semi-dinámicas y cómo es el mecanismo que se utiliza para almacenar los valores de retorno en el registro de activación.
- b) (Pts. 10) Indique cómo es la inicialización por defecto de las variables en lenguaje C al momento de su alocaión en memoria.
- c) (Pts. 5) ¿Qué diferencias hay entre ligadura estática y dinámica? Defina el concepto de ligadura (para los distintos atributos de las variables tomando como ejemplo al lenguaje C).

b)



B



1) a) $G = \{N, T, S, P\}$

$N = \{ \langle \text{funcionC} \rangle, \langle \text{tipo} \rangle, \langle \text{identificador} \rangle, \langle \text{sentencia} \rangle, \langle \text{retorno} \rangle, \langle \text{letra} \rangle, \langle \text{caracter} \rangle, \langle \text{digito} \rangle, \langle \text{sentencia_if} \rangle, \langle \text{sentencia_for} \rangle, \langle \text{sentencia_while} \rangle, \langle \text{llamado_funcion} \rangle \}$

$T = \{ "(", ")", "{", "}", "int", "void", "double", "string", "static", "...", "-", "a", "b", ..., "z", "A", "B", ..., "Z", "0", "1", ..., "9", "return" \}$

$S = \langle \text{funcionC} \rangle$

Si hay solo un parametro no lleva "

$P = \{ \langle \text{funcionC} \rangle ::= \langle \text{tipo} \rangle \langle \text{identificador} \rangle (" \{ \langle \text{tipo} \rangle \langle \text{identificador} \rangle, \dots \} " \{ \langle \text{sentencia} \rangle \}^+)$

$[\langle \text{retorno} \rangle] " \}$

$\langle \text{tipo} \rangle ::= ("int" | "void" | "double" | "string" | \dots | \langle \text{identificador} \rangle | \dots)$

$[\text{Static}]$

$\langle \text{identificador} \rangle ::= \langle \text{letra} \rangle | "-" | \{ \langle \text{caracter} \rangle \}^*$

$\langle \text{caracter} \rangle ::= \langle \text{letra} \rangle | "-" | \langle \text{digito} \rangle$

$\langle \text{letra} \rangle ::= (a | \dots | z | A | \dots | Z)$

$\langle \text{digito} \rangle ::= (0 | 1 | \dots | 9)$

$\langle \text{sentencia} \rangle ::= (\langle \text{sentencia_if} \rangle | \langle \text{sentencia_for} \rangle | \langle \text{sentencia_while} \rangle | \langle \text{sentencia_llamado_funcion} \rangle$

$| \langle \text{funcionC} \rangle | \dots)$

$\langle \text{retorno} \rangle ::= "return" (\langle \text{identificador} \rangle | \{ \langle \text{digito} \rangle \}^+)$

}

B-

3) a) *1 ***Reg Activ MiPrograma
PR
LE
LD
i = 3
x = 5
~~dec~~
Procedure tres
Procedure dos
Procedure uno
VR
*2 ***Reg Activ Uno
PR
LE(*1)
LD(*1)
i = 1

→ Está antes el punto 3

independientemente de su i-valor.

4) a) b) En C las variables globales y variables estáticas se ~~de~~ inicializan por default. La diferencia clave entre estas es su ubicación en memoria. Las variables estáticas ~~se~~ se alojan en memoria de código mientras que las globales en la stack. Cabe mencionar que las variables locales no inicializan por default.

esta ligadura estática.

a) Las variables semiautómicas en memoria de datos. Un ejemplo podrían ser las variables de tipo vector en Ada con rango de 0 a N, N siendo una constante numérica tipada. *También la parte dinámica?* *no necesariamente*

El mecanismo que se utiliza para almacenar los valores de retorno en un registro de activación es así: solo si se invoca una función el valor de retorno se anota en el registro de activación de ~~por~~ quien ~~se la~~ ~~llamó~~ ~~llamó~~ a la función y no en el registro de activación de la función llamada, de no ser así los valores ~~se~~ se perderían porque una vez finalizada la subrutina, el registro se desaloca de memoria.

c) Ligadura: momento en el que se asocia el r-valor con el l-valor.

La ligadura estática es realizada antes de la ejecución mientras que la ligadura dinámica es en ejecución.

Ejemplos en C:

R - int a → Se liga el tipo ^{int} con la variable a en compilación
static int a = 0 → Se liga el valor ^{0 (l-valor)} por default en compilación.

int a = 1
int a = 1 + 2 } El r-valor cambia en ejecución → ligadura dinámica.

Falta explicar el concepto de ligadura para cada atributo de la variable

3) a)

*1	***Rep Activ. Mi Programa PR LE LD $i = 8 - 2 - 4 - 1 = 6$ $x = 8 - 4$ $a(1) = 2$ $a(2) = 4$ $a(3) = 6 - 8$ $a(4) = 8$ $a(5) = 10$ $a(6) = 12$ Procedure Tres Procedure Dos Procedure Uno VR
*2	***Rep Activ. Uno PR LE(*1) LD(*1) $i = 1 + 6 - 6$ VR
*3	***Rep Activ. Dos PR LE(*1) LD(*2) $x = 7$ Function fun VR 8
*4	***Rep Activ. fun PR LE(*3) LD(*3) PR
*5	***Rep Activ. Tres PR LE(*1) LD(*2)

la variable
cuando hago 1...6 me refiero a que el valor
valor el valor 1, luego el 2, luego 3, hasta el 6 incluido.

Imprime: 4 → var global "i"
4 → var global "x"
2 → a(1)
4 → a(2)
8 → a(3)
8 → a(4)
10 → a(5)
12 → a(6)

Imprime: 6 → write(i)

B

b)

Registro de Activación	→ Representa los valores y atributos de un bloque (procedimiento, main, función) en forma de pila.
Punto de Retorno (PR)	→ Dirección de memoria ^{memoria} donde se guarda el punto de partida cuando al rep. se asocia, en el normal del programa.
Link Estático (LE)	→ Tiene que ver con cómo está contenido el registro de activación en base a la estructura del programa.
Link Dinámico (LD)	→ ^{con} El registro de que registro de ^{se} encuentra Hace referencia al reg. de activación que lo invocó. Ligando a la ejecución del programa.
Variables	→ Son las variables locales al registro de activación. Toman cualquier tipo y se representa la modificación de su valor en la pila.
Parámetros	
Procedures	→ Procedimientos declarados dentro del bloque del registro de activación.
Functions	→ Funciones declaradas dentro del bloque del registro de activación.
Valor de Retorno (VR)	→ Si se llama a una función dentro de la lógica del bloque, el valor retornado por dicha función es almacenado aquí, o se asigna ^{se} asigna el que la invocó. en

B